

FestiveMetrics

Gestión de la Información · Grupo 5 · EPS-I

Exposición 10 min · 7 personas · hora Europe/Madrid · «Una pulsera, una API y un panel que decide contigo.»

Guión sugerido (pedagógico): lo oficial son las diapositivas; ajustad el texto a vuestras palabras. Reparto y tiempos orientativos (~10:00). Las marcas en color son señales de actuación, no se leen.

Raúl Pérez Ruiz · Apertura · Diapos 1–2 · ~1:10

DIAPO 1 · Portada · 0:20

Buenas tardes. Somos el Grupo 5 de Gestión de la Información y os presentamos **FestiveMetrics**: un sistema web para gestionar el personal de un festival en tiempo real con pulseras NFC. Toda la idea cabe en una frase: **verde, ámbar, rojo, el semáforo que lo cuenta todo de un vistazo**.

DIAPO 2 · El problema · 0:50

Partimos de un problema real. En un festival hay miles de asistentes, decenas de zonas —barras, accesos, pista, VIP, enfermería— y aforos que cambian minuto a minuto. Hoy el coordinador decide a ojo, y eso provoca tres dolores. **Visibilidad nula**: no sabe en tiempo real cuánta gente ni cuánto personal hay en cada zona. **Reacción tardía**: los déficits se detectan cuando ya hay colas, quejas o un problema de seguridad. Y **falta de datos del turno**: las horas trabajadas se calculan a mano, tarde y con errores. FestiveMetrics ataca esos tres puntos.

→ Te paso la palabra, Antonio.

Antonio García Gómez · Solución y mapa de calor · Diapos 3–4 · ~1:55

DIAPO 3 · La solución · 1:00

Gracias, Raúl. La solución son **tres capas conectadas**. La primera es el hardware: una pulsera NFC y un lector PN532 sobre un microcontrolador ESP32; al acercar la pulsera, el lector obtiene su **UID** —su identificador único— y el ESP32 lo envía. La segunda es la **API en Laravel**: el ESP32 hace una petición HTTP directa al endpoint POST /api/fichar, con el UID y un token. Y la tercera es el **dashboard**: mapa de calor, panel de recomendaciones, mensajería e informes. El stack de la demo es Laravel 13 sobre PHP 8.3, base de datos SQLite, Blade más TailwindCSS en el front, el firmware en ESP-IDF y toda la hora sellada en Europe/Madrid.

DIAPO 4 · Zonas · mapa de calor · 0:55

Y aquí arranca la historia. Mostramos el recinto como un **mapa de calor con semáforo**: 19 zonas, cada una con su tipo, su aforo y las personas que se desean en ella. El código es universal y no necesita explicación: verde, todo bien; ámbar, precaución, una zona que conviene vigilar; rojo, situación crítica, hay que actuar ya. De un golpe de vista el coordinador ya sabe dónde mirar.

→ Alberto, ¿cómo decide el sistema qué es verde y qué es rojo?

Alberto Romero Corredor · Parametrización y déficits · Diapos 5–6 · ~2:00

DIAPO 5 · Parametrización (PUNTO CLAVE) · 1:05

Este es, para nosotros, el **punto clave**, y es justo lo que nos pidió el profesor Llopis: que las recomendaciones no estén escritas a fuego, sino que se calculen con **reglas configurables**. La regla actual es sencilla: un trabajador por cada 50 metros cuadrados, más un trabajador por cada 100 clientes, con un mínimo de uno. Lo importante es que esos umbrales —los 50 metros, los 100

clientes— los **define el administrador** y puede ajustarlos por zona. Si el personal actual cae por debajo del recomendado, salta el aviso de déficit. A la derecha veis la comparación: personal recomendado frente a actual, zona por zona.

DIAPO 6 · Panel · recomendación · 0:55

Ese aviso se vuelve accionable en el panel. Pinchas una zona: a la izquierda, qué pasa; a la derecha, la recomendación. Por ejemplo la **Zona F**: aforo 120, 240 personas dentro, tres camareros actuales frente a ocho recomendados, en rojo. El sistema propone, literalmente: faltan cinco camareros, cógelos de la Barra M o de la Barra Tesoro, que están sobradas. El criterio combina **cercanía entre zonas** —para no cruzar a alguien todo el recinto— y **balance de personal**. Con un clic en «Aceptar y reasignar» se mueve al trabajador y se le avisa.

→ Lo veis en directo. Andrés.

Andrés Senabre González · Demo en vivo · Diapo 7 · ~1:40

» **CAMBIO A NAVEGADOR** — web ya abierta y logueada como admin antes de empezar.

DIAPO 7 · Demo en vivo · situación dramática · 1:40

Vamos a verlo sobre el dashboard, en cinco pasos. **Uno**: llega un turno, varios trabajadores fichan con la pulsera y el panel de fichajes recientes se actualiza. **Dos**: salta el déficit; mirad cómo la Zona F pasa a **rojo** en el mapa porque hay menos personal del recomendado. **Tres**: aviso al admin; al entrar en la zona aparece la recomendación de mover gente desde una barra sobrada. **Cuatro**: reasignación; un solo clic mueve al trabajador y genera el mensaje automático para avisarle. Y **cinco**: zona estabilizada, la Zona F vuelve a **verde**. Ese es el corazón del sistema: detectar, recomendar y resolver, sin adivinar nada.

» **VUELTA A LAS DIAPOS. Plan B si falla conexión/proyector: capturas de los 5 pasos preparadas.**

→ WenTao, ¿qué pasa por debajo cuando alguien ficha?

WenTao Hu · Fichaje NFC por dentro · Diapo 8 · ~0:55

DIAPO 8 · Registro NFC · 0:55

Gracias, Andrés. ¿Cómo se convierte un toque en un fichaje? La clave es que **la pulsera no decide nada: solo manda su UID**, y toda la lógica vive en la API. La petición es un POST a /api/fichar con dos cosas: el UID y un **token compartido**. Si el UID es conocido, el sistema **alterna el turno** —funciona como un interruptor—: si había un turno abierto, lo cierra y calcula las horas; si no, abre uno nuevo. Tres detalles que lo sostienen: el token de cada petición se valida contra el del firmware, así que sin token no hay fichaje; los tiempos se sellan en hora Europe/Madrid; y un UID desconocido se cachea 30 segundos para autocompletar el alta de esa pulsera nueva.

→ Victoria, la coordinación.

Victoria María Agarici · Mensajería e incidencias · Diapo 9 · ~0:45

DIAPO 9 · Coordinación · 0:45

Gracias. La coordinación funciona en los **dos sentidos**. La dirección manda órdenes con tres modos de mensajería: **individual** a un trabajador concreto, **por grupo o zona** a todo un rol o a los de una zona, y **contextual**, adaptando el mensaje al rol, la zona y la ocupación. Y cualquier usuario reporta lo que ve: las incidencias son de **reporte abierto**, las envía cualquier usuario registrado, tienen estado pendiente o resuelta y se cuentan en el informe. Todo desemboca en un **panel único** del administrador, en tiempo real.

→ Sergio cierra con los datos del día.

DIAPO 10 · Estadísticas · 0:55

Al cerrar la jornada, todo se resume en datos. Tres cosas: las **horas trabajadas** por empleado, calculadas desde sus fichajes; el **recorrido**, por qué zonas pasó cada uno; y la **eficacia por hora**, los ingresos de la zona divididos entre los trabajadores activos. Un ejemplo: cinco trabajadores que ingresan cien euros en una hora dan veinte euros por trabajador y hora. Y el **ratio empleado a cliente**, por ejemplo uno a cuarenta, que se puede ordenar y comparar por zona y por persona.

DIAPO 11 · Mantenimiento · 0:45

Y está pensado para aguantar el día real del festival: **ocho casuísticas** documentadas con síntomas, causas y plan de acción; las críticas son espacio en disco, caída del backend, pérdida de datos y brecha de seguridad. Más un **protocolo de escalado** por gravedad: un crítico, P1, en menos de 30 minutos; un P2 alto, en menos de 2 horas; un P3 medio, en menos de 8; y un P4 bajo, en menos de 72.

→ Cerramos.

Cierre (lo abre quien abrió: Raúl) · Diapos 12–13 · ~0:40

DIAPO 12 · El equipo · 0:25

Somos siete personas y un solo sistema: Raúl en backend y la API de fichaje, Antonio en el mapa de calor y las zonas, Alberto en parametrización y déficits, Victoria en mensajería e incidencias, Sergio en reporte y estadísticas, WenTao en el firmware ESP32 y NFC, y Andrés en el diseño de la interfaz y la demo.

DIAPO 13 · Cierre · 0:15

Y con esto cerramos: **FestiveMetrics, el festival bajo control y en tiempo real**. Muchas gracias. ¿Preguntas?

Anexo · Preguntas probables del tribunal (chuleta de defensa)

Respuestas en corto para los 5 minutos de preguntas. Las marcadas con ★ son las trampas más fáciles de provocar.

★ ¿Qué base de datos usáis?

En la demo, **SQLite** con datos de ejemplo. El plan de mantenimiento contempla Oracle en producción; la migración es directa porque usamos Eloquent, que abstrae el motor.

★ El documento habla de Arduino y un puente en Python. ¿Qué pasó?

Fue la primera versión. Evolucionamos a un **ESP32 con firmware ESP-IDF** que hace la petición HTTP directa, quitando el intermediario y un punto de fallo.

¿La ocupación es real o simulada?

Hoy **simulada** (valor aleatorio por zona, cacheado una hora). Está preparada para una fuente real de conteo; el profesor autorizó inventar datos para la demo.

¿Cómo calculáis el personal necesario?

Regla configurable: $\text{floor}(\text{m}^2/50) + \text{floor}(\text{clientes}/100)$, con mínimo 1. Los umbrales los fija el administrador.

¿Y la eficacia por hora?

Ingresos de la zona / trabajadores activos. Ejemplo: 100 € entre 5 = 20 € por trabajador y hora.

¿Cómo evitáis fichajes falsos?

Token compartido validado en el servidor (sin token no se ficha), web con login por sesión y datos aislados por usuario. El plan añade rate limiting, HTTPS y 2FA.

¿Qué pasa si falla el lector NFC en pleno festival?

Casuística C-03: reintentos automáticos, modo degradado con cola local para sincronizar después y hardware de repuesto. Objetivo < 30 min.

¿Es «tiempo real» de verdad?

El panel se refresca consultando fichajes recientes y métricas cacheadas; no es streaming por websockets, pero para la operativa es suficiente. Queda como mejora futura.

¿Y si alguien olvida fichar la salida?

Funciona por toggle: al volver a pasar la pulsera se cierra el turno. Mientras, queda «abierto» y visible en el panel.

¿Por qué NFC y no QR o huella?

Instantáneo de un toque, barato, robusto en exterior, no depende de smartphone ni cobertura y no guarda biometría sensible.

Cifras para no titubear: 19 zonas · 200+ trabajadores · 13 roles · 8 casuísticas. Stack real: Laravel 13 · PHP 8.3 · SQLite · Blade+Tailwind · ESP32+PN532. Hora: Europe/Madrid.